

## Praktikum 12 – Machine Learning Principal Component Analysis (PCA)

Prepared By:

Dr. Sirojul Munir S.Si,M.Kom

Diah Ayu Puspasari

Elyas Randi Renaldi

Tujuan : Menjelaskan konsep dasar Principal Component Analysis (PCA) dan mengimplementasikannya untuk reduksi dimensi pada dataset Sirup, mulai dari tahap preprocessing, transformasi PCA, hingga penerapan model klasifikasi SVM dan evaluasi kinerja model sebelum dan sesudah PCA.

Dateline : 1 Pekan

Gitlab/Github :

Branch Repository : [PRODI ROMBEL]\_[NAMASINGKAT]\_[NIM] ( contoh: ti01\_budi\_0110112001)

### Aturan Pengerjaan:

1. Gunakan text editor yang nyaman bagi anda
2. Diperkenankan mengerjakan langsung bagi yang sudah memahami dan menguasai materi
3. Dilarang melakukan tindakan plagiarism (asisten lab akan mengecek hasil pekerjaan)
  - a. 1x nilai praktikum terkait bernilai 0
  - b. 2x nilai matakuliah pemrograman web E
  - c. 3x mahasiswa akan di sidang komite etik kampus

## Pendahuluan

Untuk mengatasi hal overfitting — yaitu kondisi di mana model terlalu “menghafal” data latih dan kehilangan kemampuan generalisasi terhadap data baru, gunakanlah pendekatan Reduksi Dimensi (Dimensionality Reduction), yaitu proses mengurangi jumlah fitur dalam dataset tanpa kehilangan informasi penting di dalamnya. Tujuan utama reduksi dimensi adalah:

- Menyederhanakan representasi data,
- Mengurangi kompleksitas komputasi,
- Menghindari overfitting,
- Serta meningkatkan kemampuan visualisasi data berdimensi tinggi.

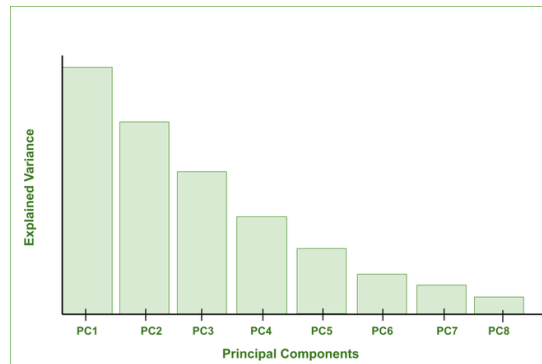
Salah satu metode reduksi dimensi yang paling populer adalah Analisis Komponen Utama (Principal Component Analysis / PCA). PCA merupakan teknik ekstraksi fitur (feature extraction) yang memetakan ruang fitur berdimensi tinggi ke ruang fitur berdimensi lebih rendah.

Teknik ini bekerja dengan mencari arah (komponen utama) yang menjelaskan variansi terbesar dalam data — artinya, PCA mempertahankan sebanyak mungkin informasi dari dataset asli dalam jumlah dimensi yang lebih sedikit.

Setiap Komponen Utama (Principal Component / PC) merupakan kombinasi linear dari fitur asli, dengan urutan:

- PC1: menjelaskan variansi data terbesar,
- PC2: menjelaskan variansi terbesar kedua, dan seterusnya.

Hubungan antar komponen ini bersifat tidak berkorelasi (orthogonal), sehingga informasi antar dimensi baru menjadi independen.



Gambar atau grafik batang yang menampilkan “Explained Variance Ratio” menunjukkan proporsi informasi yang dijelaskan oleh tiap komponen utama — semakin besar nilai variansi yang dijelaskan, semakin banyak informasi dari data asli yang berhasil dipertahankan oleh PCA.

Dalam konteks Machine Learning, PCA memiliki dua manfaat utama:

- Untuk Visualisasi Data — misalnya, mereduksi data 4 dimensi (seperti dataset Iris) menjadi 2 atau 3 dimensi agar dapat divisualisasikan dengan mudah.
- Untuk Optimasi Model — mengurangi jumlah fitur agar proses pelatihan model seperti SVM atau Logistic Regression menjadi lebih cepat dan efisien, tanpa kehilangan akurasi secara signifikan.

### Langkah Awal:

Sebelum memulai praktikum ini, pastikan Anda telah menyiapkan struktur folder di Google Drive / Jupyter seperti pada praktikum sebelumnya. Untuk praktikum ke-12, buat sub-folder baru dengan nama praktikum12/ di dalam direktori utama praktikum\_ml/.

Struktur folder yang digunakan adalah sebagai berikut:

```
praktikum_ml/  
├─ praktikum01/  
├─ praktikum02/  
├─ praktikum03/  
│   └─ data/  
│   └─ notebooks/  
│   └─ model/  
│   └─ reports/
```

#### Catatan:

1. Jika telah memiliki folder praktikum\_ml, cukup tambahkan folder praktikum12.
2. Semua file notebook pada praktikum ini disimpan di dalam folder notebooks/ dengan nama praktikum12.ipynb.
3. Dataset ditempatkan di folder data/.
4. Model hasil training disimpan di folder model/ (format .pkl atau .joblib).
5. Visualisasi atau laporan hasil analisis disimpan di folder reports/.Jalankan file installer yang sudah diunduh.

Setelah itu, lakukan Langkah-langkah seperti biasa untuk Loading dataset. Mulai dari menghubungkannya dengan google drive sampai membaca dataset.

## Persiapan Praktikum PCA (Principal Component Analysis)

### 1. Import Library Dasar

Pada tahap ini, dilakukan proses persiapan lingkungan pemrograman dengan mengimpor pustaka (library) Python yang dibutuhkan untuk melakukan analisis data, reduksi dimensi dengan PCA, serta pelatihan model Machine Learning menggunakan algoritma SVM (Support Vector Machine).

```
# Import library dasar  
import numpy as np  
import pandas as pd  
  
from sklearn.datasets import load_wine  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
  
import matplotlib.pyplot as plt  
from mpl_toolkits.mplot3d import Axes3D # untuk 3D plot  
# Agar plot tampil rapi  
plt.rcParams['figure.figsize'] = (8, 6)
```

- `from sklearn.datasets import load_wine`: Memanggil dataset Sirup bawaan dari pustaka scikit-learn. Dataset ini berisi 13 fitur kimia dari berbagai jenis anggur yang diklasifikasikan ke dalam 3 kategori.

- `from sklearn.model_selection import train_test_split`: Digunakan untuk membagi dataset menjadi data latih (train) dan data uji (test) agar model dapat dievaluasi dengan benar.
- `from sklearn.preprocessing import StandardScaler`: Melakukan standardisasi fitur sehingga setiap kolom memiliki rata-rata 0 dan standar deviasi 1. Standardisasi ini penting sebelum melakukan PCA karena PCA sensitif terhadap skala data.
- `from sklearn.decomposition import PCA`: Pustaka utama untuk melakukan Principal Component Analysis (PCA), yaitu proses reduksi dimensi dari 13 fitur menjadi 3 komponen utama.
- `from sklearn.svm import SVC`: Mengimpor Support Vector Classifier (SVC), model klasifikasi berbasis SVM yang digunakan untuk membangun model dan melakukan prediksi setelah PCA diterapkan.
- `from sklearn.metrics import accuracy_score, classification_report, confusion_matrix`: Berfungsi untuk mengukur performa model, meliputi tingkat akurasi, precision, recall, dan visualisasi confusion matrix.
- `import matplotlib.pyplot as plt` dan `from mpl_toolkits.mplot3d import Axes3D`: Digunakan untuk membuat visualisasi 2D dan 3D dari hasil PCA.
  - `matplotlib.pyplot` berfungsi untuk membuat grafik dasar, sedangkan `Axes3D` digunakan khusus untuk memplot data dalam ruang tiga dimensi (3 komponen PCA).
- `plt.rcParams['figure.figsize'] = (8, 6)`: Mengatur ukuran default dari grafik yang akan ditampilkan agar visualisasi hasil PCA tampak proporsional dan mudah dibaca.

## 2. Load Dataset Sirup

Setelah proses instalasi library tambahan selesai, tahap selanjutnya adalah melakukan import seluruh. Pada tahap ini dilakukan proses pembacaan (loading) dataset Sirup yang sudah disediakan oleh pustaka scikit-learn. Dataset ini berisi hasil analisis kimia dari tiga jenis anggur yang berbeda, dengan total 178 data sampel dan 13 fitur kimia sebagai variabel prediktor.

Fitur-fitur tersebut meliputi antara lain kadar alkohol, magnesium, fenol, intensitas warna, hingga kandungan proline. Tujuan dari pemuatan dataset ini adalah untuk memahami struktur data dan memastikan bahwa dataset siap digunakan untuk proses reduksi dimensi dengan PCA.

```
# Load dataset Wine
wine = load_wine()

df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['target'] = wine.target

# Tampilkan 5 baris pertama
df.head()
```

|   | alcohol | malic_acid | ash  | alcalinity_of_ash | magnesium | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | color_intensity | hue  | od280/od315_of_diluted_wines | proline | target |
|---|---------|------------|------|-------------------|-----------|---------------|------------|----------------------|-----------------|-----------------|------|------------------------------|---------|--------|
| 0 | 14.23   | 1.71       | 2.43 | 15.6              | 127.0     | 2.80          | 3.06       | 0.28                 | 2.29            | 5.64            | 1.04 | 3.92                         | 1065.0  | 0      |
| 1 | 13.20   | 1.78       | 2.14 | 11.2              | 100.0     | 2.65          | 2.76       | 0.26                 | 1.28            | 4.38            | 1.05 | 3.40                         | 1050.0  | 0      |
| 2 | 13.16   | 2.36       | 2.67 | 18.6              | 101.0     | 2.80          | 3.24       | 0.30                 | 2.81            | 5.68            | 1.03 | 3.17                         | 1185.0  | 0      |
| 3 | 14.37   | 1.95       | 2.50 | 16.8              | 113.0     | 3.85          | 3.49       | 0.24                 | 2.18            | 7.80            | 0.86 | 3.45                         | 1480.0  | 0      |
| 4 | 13.24   | 2.59       | 2.87 | 21.0              | 118.0     | 2.80          | 2.69       | 0.39                 | 1.82            | 4.32            | 1.04 | 2.93                         | 735.0   | 0      |

Next steps: [Generate code with df](#) [New interactive sheet](#)

- `pd.DataFrame(wine.data, columns=wine.feature_names)`
  - Mengubah data menjadi format DataFrame dari pustaka pandas agar lebih mudah dibaca dan dianalisis.
  - Parameter `wine.feature_names` berfungsi memberikan nama kolom sesuai daftar fitur dalam dataset.

- `df['target'] = wine.target`
  - Menambahkan kolom baru bernama target yang berisi label kelas dari jenis Sirup (0, 1, atau 2).
  - Masing-masing label mewakili kategori Sirup yang berbeda.

### 3. Eksplorasi Awal Dataset Sirup

Tahap ini bertujuan untuk memahami struktur data yang digunakan sebelum dilakukan proses reduksi dimensi. Analisis eksplorasi awal (Exploratory Data Analysis / EDA) mencakup pemeriksaan tipe data, jumlah data, nilai kosong, data duplikat, serta distribusi label (target class).

Langkah ini membantu memastikan bahwa dataset dalam kondisi bersih dan siap untuk diproses lebih lanjut.

```
# Informasi umum dataset
df.info()

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   alcohol                               178 non-null    float64
1   malic_acid                           178 non-null    float64
2   ash                                   178 non-null    float64
3   alcalinity_of_ash                    178 non-null    float64
4   magnesium                            178 non-null    float64
5   total_phenols                        178 non-null    float64
6   flavanoids                           178 non-null    float64
7   nonflavanoid_phenols                 178 non-null    float64
8   proanthocyanins                      178 non-null    float64
9   color_intensity                      178 non-null    float64
10  hue                                   178 non-null    float64
11  od280/od315_of_diluted_wines         178 non-null    float64
12  proline                              178 non-null    float64
13  target                                178 non-null    int64
dtypes: float64(13), int64(1)
memory usage: 19.6 KB
```

Perintah `df.info()` digunakan untuk menampilkan informasi struktur dataset, seperti jumlah baris (entries), nama kolom, jumlah data non-null, serta tipe data pada setiap kolom.

```
# Statistik deskriptif
df.describe()
```

|       | alcohol    | malic_acid | ash        | alcalinity_of_ash | magnesium  | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | color_intensity | hue        | od280/od315_of_diluted_wines |
|-------|------------|------------|------------|-------------------|------------|---------------|------------|----------------------|-----------------|-----------------|------------|------------------------------|
| count | 178.000000 | 178.000000 | 178.000000 | 178.000000        | 178.000000 | 178.000000    | 178.000000 | 178.000000           | 178.000000      | 178.000000      | 178.000000 | 178.000000                   |
| mean  | 13.000618  | 2.336348   | 2.366517   | 19.494944         | 99.741573  | 2.295112      | 2.029270   | 0.361854             | 1.590899        | 5.058090        | 0.957449   | 2.611685                     |
| std   | 0.811827   | 1.117146   | 0.274344   | 3.339564          | 14.282484  | 0.625851      | 0.998859   | 0.124453             | 0.572359        | 2.318286        | 0.228572   | 0.709990                     |
| min   | 11.030000  | 0.740000   | 1.360000   | 10.600000         | 70.000000  | 0.980000      | 0.340000   | 0.130000             | 0.410000        | 1.280000        | 0.480000   | 1.270000                     |
| 25%   | 12.362500  | 1.602500   | 2.210000   | 17.200000         | 88.000000  | 1.742500      | 1.205000   | 0.270000             | 1.250000        | 3.220000        | 0.782500   | 1.937500                     |
| 50%   | 13.050000  | 1.865000   | 2.360000   | 19.500000         | 98.000000  | 2.355000      | 2.135000   | 0.340000             | 1.555000        | 4.690000        | 0.985000   | 2.780000                     |
| 75%   | 13.677500  | 3.082500   | 2.557500   | 21.500000         | 107.000000 | 2.800000      | 2.875000   | 0.437500             | 1.950000        | 6.200000        | 1.120000   | 3.170000                     |
| max   | 14.830000  | 5.800000   | 3.230000   | 30.000000         | 162.000000 | 3.880000      | 5.080000   | 0.660000             | 3.580000        | 13.000000       | 1.710000   | 4.000000                     |

Fungsi `df.describe()` menampilkan rangkuman statistik deskriptif dari setiap kolom numerik.

```
# Cek missing value
df.isnull().sum()

***

```

|                              | 0 |
|------------------------------|---|
| alcohol                      | 0 |
| malic_acid                   | 0 |
| ash                          | 0 |
| alcalinity_of_ash            | 0 |
| magnesium                    | 0 |
| total_phenols                | 0 |
| flavanoids                   | 0 |
| nonflavanoid_phenols         | 0 |
| proanthocyanins              | 0 |
| color_intensity              | 0 |
| hue                          | 0 |
| od280/od315_of_diluted_wines | 0 |
| proline                      | 0 |
| target                       | 0 |

```
dtype: int64
```

Perintah ini digunakan untuk memeriksa apakah terdapat **nilai kosong (missing value)** pada dataset.

```
# Cek data duplikat
df.duplicated().sum()

np.int64(0)
```

Fungsi ini digunakan untuk mendeteksi apakah terdapat baris data yang identik di dataset.

```
# Distribusi kelas target
print("Nama kelas:", wine.target_names)
df['target'].value_counts()

Nama kelas: ['class_0' 'class_1' 'class_2']

```

|        | count |
|--------|-------|
| target |       |
| 1      | 71    |
| 0      | 59    |
| 2      | 48    |

```
dtype: int64
```

Distribusi label target cukup seimbang, memungkinkan penerapan algoritma klasifikasi seperti SVM tanpa perlu melakukan penyeimbangan data tambahan.

#### 4. Pemisahan Fitur dan Label

Tahap ini bertujuan untuk memisahkan data fitur (X) dan label target (y) dari dataset.

Fitur (X) berisi nilai-nilai variabel independen (dalam hal ini fitur kimia pada Sirup), sedangkan label (y) berisi variabel dependen atau kelas yang akan diprediksi (jenis Sirup).

```
# Fitur dan label
X = wine.data          # bisa juga: df[wine.feature_names].values
y = wine.target        # df['target'].values

print("Shape X:", X.shape) # (178, 13)
print("Shape y:", y.shape) # (178,)

Shape X: (178, 13)
Shape y: (178,)
```

- `X = wine.data`
  - Menyimpan seluruh fitur numerik dari dataset Sirup ke dalam variabel X.
  - Dataset memiliki 13 fitur kimia, seperti kadar alkohol, fenol, magnesium, dll.
  - Dengan demikian, X memiliki bentuk (178, 13) → artinya ada 178 baris data dan 13 kolom fitur.
- `y = wine.target`
  - Menyimpan kolom label target ke dalam variabel y.
  - Kolom ini berisi nilai 0, 1, atau 2, yang merepresentasikan tiga jenis Sirup berbeda.
  - Bentuknya (178,) karena hanya satu kolom target untuk setiap baris data.

## 5. Pembagian Data Latih dan Data Uji (Train-Test Split)

Setelah data fitur (X) dan label (y) dipisahkan, tahap berikutnya adalah membagi dataset menjadi data latih (training data) dan data uji (testing data).

```
# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Shape X_train:", X_train.shape)
print("Shape X_test :", X_test.shape)

Shape X_train: (142, 13)
Shape X_test : (36, 13)
```

Langkah ini bertujuan agar model Machine Learning dapat dilatih pada sebagian data, kemudian dievaluasi pada data lain yang belum pernah dilihat sebelumnya, sehingga hasil pengujian lebih objektif dan tidak bias (overfitting prevention).

## 6. Standardisasi Data (Mean = 0, Std = 1)

Sebelum melakukan Principal Component Analysis (PCA), setiap fitur pada dataset harus melalui proses standardisasi.

Tujuannya adalah untuk memastikan bahwa seluruh fitur berada pada skala yang sebanding, sehingga tidak ada fitur yang mendominasi hasil komponen utama hanya karena memiliki nilai numerik yang lebih besar.

```
# Standardisasi (mean=0, std=1)
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train) # fit + transform pada data train
X_test_scaled = scaler.transform(X_test)      # hanya transform pada data test

X_train_scaled[:5]
```

```
... array([[ 0.38580089, -0.63787118,  1.77666817, -1.22453161,  0.69643032,
            0.52686525,  0.73229212, -0.1695489 , -0.41578344, -0.16746725,
            0.62437819,  0.2529082 ,  0.46772474],
          [ 0.94851892, -0.76544542,  1.25317383,  0.85328406,  0.09178497,
            1.17279546,  1.33318146, -0.59045701,  1.34974202,  0.30530313,
            1.06715537,  0.15104809,  1.81576773],
          [ 0.52335419, -0.51940939,  0.9540342 , -1.04643312, -0.44567755,
            0.93057163,  1.006382 , -0.1695489 , -0.26000178, -0.081509 ,
            -0.12834302,  0.89317174,  1.51620262],
          [ 0.97352861, -0.55585917,  0.16879269, -1.0761162 , -0.71440882,
            0.52686525,  0.81662747, -0.59045701,  0.36312485,  0.262324 ,
            0.8900445 ,  0.42752553,  1.93226527],
          [ 0.43582027,  0.82012009,  0.05661533,  0.55645325, -0.51286037,
            -0.55506784, -1.29175618,  0.75644894, -0.60618325,  1.47433535,
            -1.76661859, -1.43505932, -0.29783054]])
```

- `scaler.fit_transform(X_train)` untuk melakukan dua proses sekaligus:
  - `fit()` → menghitung nilai rata-rata dan standar deviasi dari data training,
  - `transform()` → menggunakan nilai tersebut untuk menstandarisasi data training.
  - Penting: proses fit hanya dilakukan pada data latih (train) agar model tidak “mengintip” data uji (data leakage).

## 7. Penerapan Model SVM tanpa PCA (Baseline Model).

Sebelum melakukan reduksi dimensi dengan PCA, terlebih dahulu dibuat model baseline menggunakan seluruh fitur asli (13 fitur).

```
# Model SVM tanpa PCA
svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

# Prediksi dan evaluasi
y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi SVM tanpa PCA:", acc_no_pca)

print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca, target_names=wine.target_names))
```

```
... Akurasi SVM tanpa PCA: 0.9722222222222222
```

```
Classification Report (tanpa PCA):
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| class_0      | 1.00      | 1.00   | 1.00     | 12      |
| class_1      | 0.93      | 1.00   | 0.97     | 14      |
| class_2      | 1.00      | 0.90   | 0.95     | 10      |
| accuracy     |           |        | 0.97     | 36      |
| macro avg    | 0.98      | 0.97   | 0.97     | 36      |
| weighted avg | 0.97      | 0.97   | 0.97     | 36      |

Model baseline ini berfungsi sebagai pembandingan (benchmark) untuk mengevaluasi apakah penerapan PCA nantinya dapat mempertahankan atau meningkatkan akurasi model klasifikasi.



Penjelasan Kode:

- `SVC(kernel='rbf', gamma='scale')`
  - Membuat model Support Vector Classifier (SVM) dengan kernel Radial Basis Function (RBF).
  - Kernel RBF digunakan untuk menangani data yang tidak linear, seperti dataset Sirup.
  - Parameter `gamma='scale'` secara otomatis menyesuaikan kompleksitas model dengan ukuran dataset.
- `svm_no_pca.fit(X_train_scaled, y_train)`
  - Melatih model SVM menggunakan data latih yang telah distandarisasi.
- `y_pred_no_pca = svm_no_pca.predict(X_test_scaled)`
  - Melakukan prediksi terhadap data uji untuk menilai performa model.

## 8. Penerapan PCA (Principal Component Analysis)

Setelah data distandarisasi, tahap selanjutnya adalah menerapkan PCA (Principal Component Analysis) untuk mereduksi jumlah fitur dari 13 menjadi 3 komponen utama.

Reduksi dimensi ini dilakukan untuk:

- menyederhanakan kompleksitas data,
- mempercepat proses pelatihan model, dan
- mempermudah visualisasi data dalam ruang berdimensi rendah (2D/3D).

```
# PCA dengan 3 komponen utama
pca = PCA(n_components=3)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Shape X_train_pca:", X_train_pca.shape) # (n_train, 3)
print("Shape X_test_pca :", X_test_pca.shape)  # (n_test, 3)

Shape X_train_pca: (142, 3)
Shape X_test_pca : (36, 3)
```

Penjelasan Kode:

- `pca = PCA(n_components=3)`
  - Membuat objek PCA dan menentukan jumlah komponen utama (principal components) sebanyak 3.
  - Artinya, dari 13 fitur awal, PCA hanya akan mengekstraksi 3 fitur baru (PC1, PC2, PC3) yang mewakili sebagian besar informasi dari data asli.
- `pca.fit_transform(X_train_scaled)`
  - `fit()` menghitung komponen utama berdasarkan data latih.
  - `transform()` mentransformasikan data latih ke ruang baru berdimensi 3 (PC1–PC3).
  - Hasilnya disimpan dalam `X_train_pca`.
- `pca.transform(X_test_scaled)`: Menerapkan transformasi yang sama pada data uji menggunakan hasil perhitungan dari data latih, sehingga tidak terjadi data leakage.

## 9. Menampilkan Variansi yang Dijelaskan oleh Setiap Komponen PCA.

Setelah PCA dilakukan, kita perlu mengetahui berapa persen informasi (variansi) dari dataset asli yang masih bisa dijelaskan oleh setiap komponen utama (Principal Component).

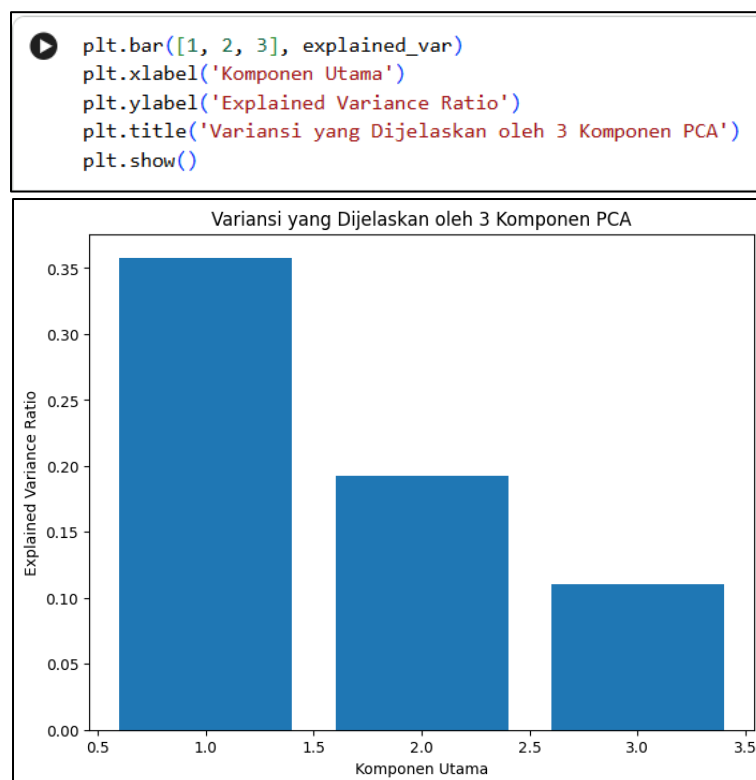
Nilai ini dikenal dengan istilah Explained Variance Ratio.

```
# Variansi yang dijelaskan oleh tiap komponen
explained_var = pca.explained_variance_ratio_
print("Explained Variance Ratio tiap komponen:", explained_var)
print("Total variansi yang dijelaskan 3 komponen pertama:", explained_var.sum())
```

Explained Variance Ratio tiap komponen: [0.35792104 0.19270671 0.11019835]  
Total variansi yang dijelaskan 3 komponen pertama: 0.6608261082211259

Output:

- PC1 menjelaskan sekitar 35.79% variansi data,
- PC2 menjelaskan sekitar 19.27%,
- PC3 menjelaskan sekitar 11.01%,
- Total ketiga komponen menjelaskan sekitar 66.08% dari informasi asli dataset.



Visualisasi Explained Variance Ratio dalam bentuk diagram batang (bar chart).

## 10. Membangun Model SVM dengan PCA

Setelah melakukan reduksi dimensi menggunakan PCA (dari 13 fitur menjadi 3 komponen utama), tahap berikutnya adalah melatih kembali model SVM menggunakan hasil transformasi PCA.

Langkah ini bertujuan untuk melihat apakah penggunaan PCA memengaruhi kinerja model klasifikasi.

```
# Model SVM dengan PCA
svm_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_pca.fit(X_train_pca, y_train)

# Prediksi dan evaluasi
y_pred_pca = svm_pca.predict(X_test_pca)

acc_pca = accuracy_score(y_test, y_pred_pca)
print("Akurasi SVM dengan PCA (3 komponen):", acc_pca)

print("\nClassification Report (dengan PCA):")
print(classification_report(y_test, y_pred_pca, target_names=wine.target_names))
```

\*\*\* Akurasi SVM dengan PCA (3 komponen): 0.9722222222222222

Classification Report (dengan PCA):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| class_0      | 1.00      | 1.00   | 1.00     | 12      |
| class_1      | 0.93      | 1.00   | 0.97     | 14      |
| class_2      | 1.00      | 0.90   | 0.95     | 10      |
| accuracy     |           |        | 0.97     | 36      |
| macro avg    | 0.98      | 0.97   | 0.97     | 36      |
| weighted avg | 0.97      | 0.97   | 0.97     | 36      |

Penjelasan Kode:

- `svm_pca = SVC(kernel='rbf', gamma='scale')`
  - Membuat model Support Vector Classifier (SVM) dengan kernel RBF (Radial Basis Function) — sama seperti model sebelumnya (baseline).
  - Penggunaan kernel yang sama memastikan hasil perbandingan lebih objektif.
- `svm_pca.fit(X_train_pca, y_train)`
  - Melatih model SVM menggunakan data hasil PCA (3 fitur utama).
  - Setiap baris data kini direpresentasikan oleh 3 komponen utama (PC1, PC2, PC3).
- `y_pred_pca = svm_pca.predict(X_test_pca)`
  - Menggunakan model yang telah dilatih untuk memprediksi kelas Sirup pada data uji yang juga telah direduksi dimensinya.
- `accuracy_score()` dan `classification_report()`
  - Mengukur performa model dari sisi akurasi, serta metrik lain seperti precision, recall, dan f1-score.

## 11. Visualisasi PCA dalam Ruang 3 Dimensi

Setelah reduksi dimensi menggunakan PCA dengan 3 komponen utama (PC1, PC2, dan PC3), kita dapat memvisualisasikan data dalam ruang tiga dimensi (3D) untuk melihat bagaimana distribusi tiap kelas (jenis Sirup) terbentuk pada ruang fitur baru.

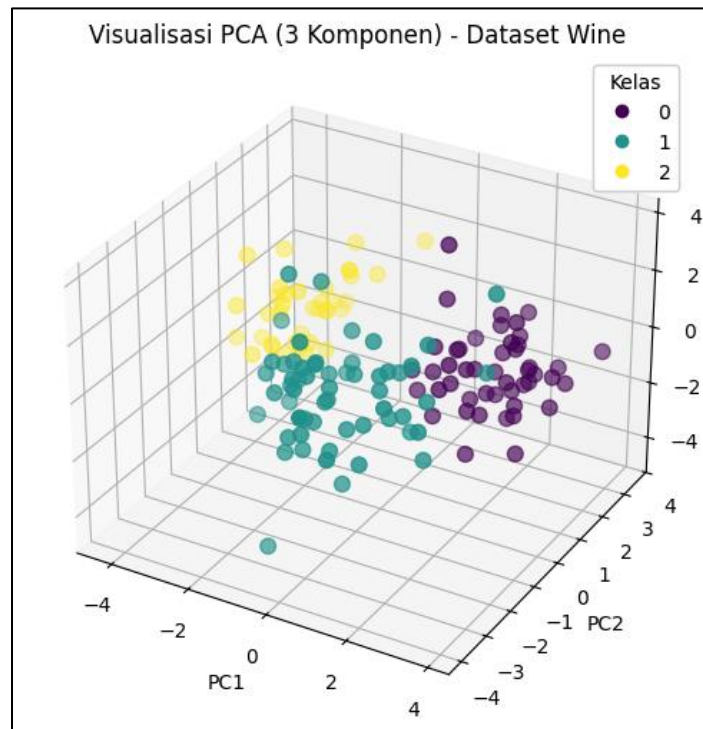
```
# Visualisasi 3D PCA (menggunakan data train)
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    X_train_pca[:, 2],
    c=y_train,
    s=60
)

ax.set_title('Visualisasi PCA (3 Komponen) - Dataset Wine')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')

# Tambahkan legend berdasarkan kelas
legend1 = ax.legend(
    *scatter.legend_elements(),
    title="Kelas"
)
ax.add_artist(legend1)

plt.show()
```



**Visualisasi 3D PCA** memperlihatkan bahwa ketiga kelas Sirup (0, 1, dan 2) dapat **terpisah dengan cukup baik** di ruang tiga dimensi hasil transformasi PCA.

## 12. Perbandingan Model SVM Tanpa PCA dan Dengan PCA

Tahapan terakhir dari praktikum ini adalah membandingkan performa model sebelum dan sesudah diterapkan PCA. Perbandingan dilakukan dari sisi:

- Jumlah fitur yang digunakan,
- Nilai akurasi model,
- dan total variansi yang dijelaskan oleh komponen PCA.

```
# Membandingkan hasil model tanpa dan dengan PCA
comparison = pd.DataFrame({
    'Model': ['SVM Tanpa PCA', 'SVM Dengan PCA (3 Komponen)'],
    'Jumlah Fitur': [X_train_scaled.shape[1], X_train_pca.shape[1]],
    'Akurasi': [acc_no_pca, acc_pca],
    'Variansi Total PCA': [None, explained_var.sum()]
})

comparison
```

|   | Model                       | Jumlah Fitur | Akurasi  | Variansi Total PCA |
|---|-----------------------------|--------------|----------|--------------------|
| 0 | SVM Tanpa PCA               | 13           | 0.972222 | NaN                |
| 1 | SVM Dengan PCA (3 Komponen) | 3            | 0.972222 | 0.660826           |

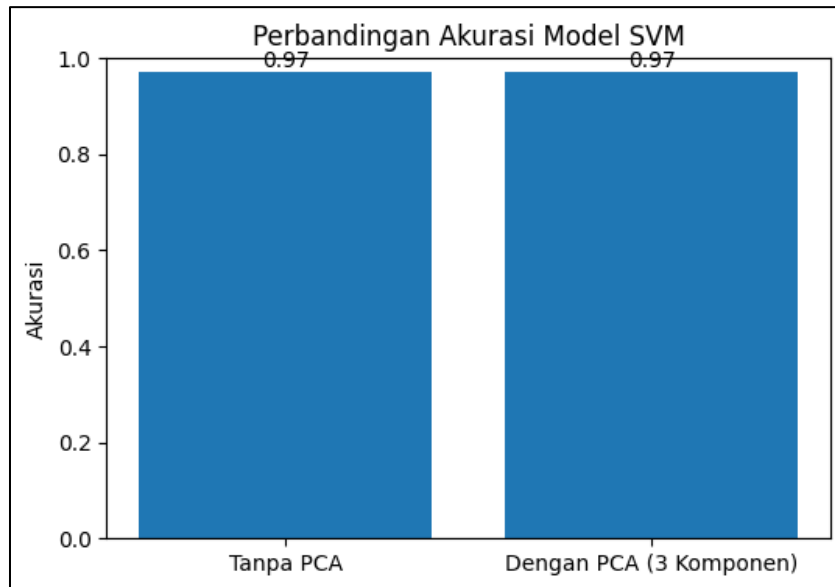
Next steps: [Generate code with comparison](#) [New interactive sheet](#)

Dari hasil disimpulkan bahwa:

- Jumlah fitur berkurang drastis dari 13 menjadi 3 komponen utama.
- Akurasi model tetap sama tinggi (97.22%), meskipun jumlah fitur direduksi hingga lebih dari 75%.
- Variansi total PCA (0.66) menunjukkan bahwa ketiga komponen utama sudah mewakili sekitar 66% informasi dari data asli.
- Hal ini membuktikan bahwa PCA dapat menyederhanakan data tanpa kehilangan performa model.

Visualisasi Grafik:

```
plt.figure(figsize=(6,4))
plt.bar(['Tanpa PCA', 'Dengan PCA (3 Komponen)'], [acc_no_pca, acc_pca])
plt.title('Perbandingan Akurasi Model SVM')
plt.ylabel('Akurasi')
plt.ylim(0, 1)
for i, v in enumerate([acc_no_pca, acc_pca]):
    plt.text(i, v + 0.01, f'{v:.2f}', ha='center')
plt.show()
```



Intepretasi Hasil Grafik:

- Kedua model (tanpa PCA dan dengan PCA) memiliki akurasi yang identik (0.97).
- Ini menunjukkan bahwa reduksi dimensi dengan PCA tidak menurunkan kinerja model SVM.
- Model dengan PCA justru lebih efisien, karena menggunakan lebih sedikit fitur dan lebih cepat dalam pelatihan.

## 14. Kesimpulan Akhir Praktikum PCA

Pada praktikum ini telah dilakukan penerapan Principal Component Analysis (PCA) sebagai salah satu teknik reduksi dimensi untuk menyederhanakan dataset tanpa mengurangi informasi penting secara signifikan. PCA diimplementasikan pada dataset Sirup, yang awalnya memiliki 13 fitur numerik dan 3 kelas target (jenis Sirup berbeda).

Dari seluruh tahapan yang dilakukan, diperoleh beberapa kesimpulan utama sebagai berikut:

1. Proses reduksi dimensi dengan PCA berhasil mengubah 13 fitur asli menjadi 3 komponen utama (PC1, PC2, dan PC3). Ketiga komponen tersebut mampu menjelaskan sekitar 66.08% variansi total dari data asli, yang berarti sebagian besar informasi masih dapat dipertahankan.
2. Model SVM tanpa PCA dan model SVM dengan PCA (3 komponen) menunjukkan akurasi yang sama tinggi, yaitu 97.22%. Hal ini membuktikan bahwa penerapan PCA tidak menurunkan performa klasifikasi meskipun jumlah fitur berkurang drastis.

3. Visualisasi 3D PCA menunjukkan bahwa data dari tiga kelas (class\_0, class\_1, dan class\_2) dapat terpisah dengan cukup baik pada ruang tiga dimensi hasil transformasi PCA, menandakan bahwa komponen utama berhasil menangkap struktur data yang relevan.
4. Dengan menerapkan PCA, model menjadi lebih efisien secara komputasi, karena bekerja dengan jumlah fitur yang lebih sedikit, namun tetap mempertahankan kinerja prediksi yang sangat baik.
5. PCA juga membantu mengurangi risiko overfitting, terutama pada dataset dengan dimensi tinggi, karena menghilangkan korelasi antar fitur dan menyederhanakan representasi data.

### Daftar Pustaka / Referensi

- [1] GeeksforGeeks. (2024). "Reduce Data Dimensionality using PCA in Python." Diakses dari: <https://www.geeksforgeeks.org/machine-learning/reduce-data-dimensionality-using-pca-python/>
- [2] BuiltIn. (2024). "PCA in Python: A Tutorial on Principal Component Analysis." Diakses dari: <https://builtin.com/machine-learning/pca-in-python>

## Tugas Praktikum Mandiri

Sebagai latihan tambahan, lakukan implementasi PCA pada dataset berikut untuk memahami penerapan PCA pada dataset medis berdimensi tinggi.

Gunakan dataset Breast Cancer dari Kaggle:

 <https://www.kaggle.com/code/kamyarazar/breast-cancer-classification-using-pca-in-python>